

Why NAVIS does not use an LLM to decide matches

NAVIS — Intelligent Data Capture & Schedule-Linking Layer | SIH 2026, Problem Statement 26122 | Status as of 11 September 2026

The short version. We could have asked a language model to read a site report and simply pick the matching schedule task. We built that first, measured it, and threw it away. The reason is simple: when a model says *"91% confident"*, that number is not real. You cannot tune it, you cannot test it, and if it is wrong it quietly writes a wrong date into a live project schedule.

What is actually running right now

Step	What does the work today	LLM?
Reading a messy daily report	Pattern rules that pull out tags, dates, quantities	No
Understanding what the words mean	MiniLM — a small neural language model, always on	Yes (small)
Choosing which schedule task it belongs to	A score built from 6 visible signals	No — on purpose
Talking to the site supervisor	Question-by-question form filling	No (switchable)

So the honest answer to *"are you using AI?"* is **yes** — a neural model reads meaning on every single event. The answer to *"is a large language model making the decision?"* is **no**, and that is a design choice, not a shortcut.

Four plain reasons we keep the model out of the decision

- 1. A model's confidence is not a real number.** "91% sure" is just words the model produced. Our score is built from things you can point at, so we can set a safety line and prove where it sits. That is the difference between a claim and a measurement.
- 2. A model cannot say "it's one of these two."** Our system compares the best answer against the second-best. If they are too close, it refuses to decide and sends it to a human. A model gives you one answer and no sense of how close the runner-up was.
- 3. A model explains itself in paragraphs, not in facts.** NAVIS explains every match with fixed labels a planner can check: *tag matched, dates line up, same discipline*. A written explanation can sound convincing and still be wrong — which is worse than a short one that is right.
- 4. A wrong answer here is silent and expensive.** A bad auto-link corrupts a project schedule and nobody notices for weeks. A review item costs a planner about ten seconds. Those two costs are nowhere near equal, so we designed for the one you cannot undo.

What we get in return

100% auto-link accuracy: zero wrong links	50.4% of updates linked with no human involved	1 command re-proves the numbers from scratch	~6 ms to match one update, not seconds
--	---	---	---

The LLM is built, tested, and one line away

Page 2 of 2 | What exists, how to turn it on, and what it is allowed to touch

Nothing is missing. The language model path is fully written, wired in and covered by tests. It ships switched off because the offline path is the one we have measured, and a demo must not depend on a model being reachable.

Turning it on takes one setting

In the configuration file, change one line:

```
EXTRACTION_PROVIDER=rules    # today: no model, no network
EXTRACTION_PROVIDER=ollama    # local model (qwen3:8b) takes over
```

It also accepts a cloud model instead of a local one. A built-in status check reports which mode the system is in, so we can never accidentally claim a model is running when it is not:

GET /agent/llm-status

The safety rails that are already in the code

Guard	What it does
5-second timeout, one try	If the model is slow or unreachable, the system falls back to the offline path silently. A model outage is never a product outage.
Everything re-checked	Every value the model returns is re-validated by our own rules before it can be used.
Tags never come from the model	Tag numbers are the strongest matching signal, so an invented one would do the most damage. They come only from pattern rules.
Dates never come from the model	A date is what actually gets written into the schedule. It is read by our own parser, never taken on trust.
The model never picks the task	Choosing the schedule activity stays with the scoring engine, always.
Words must be the supervisor's own	If the model suggests a description, at least 75% of it must already appear in what the supervisor actually said. Otherwise it is thrown away.

The rule in one sentence: the model may *suggest wording*; it may never choose a task, a tag, or a date.

What we honestly give up

Without the model switched on, very informal reports are hard for us — mixed Hindi and English ("kaam chalu hai"), references with no tag number ("the big tank"), loose quantities ("roughly 30 cum"). We keep one deliberately messy report in our test data for exactly this reason: it is the file that shows where the offline path stops and where the model earns its place. We show that file rather than hide it.

If a judge asks, the answer is one paragraph

"The language model path is built, tested and switchable. We ship with it off because the offline path is the one we have measured, and we will not make a live schedule depend on a model being reachable. A neural model still reads meaning on every event. What we removed is the model's power to *decide* — because a wrong link corrupts a schedule silently, and a review item costs a planner ten seconds."